Relatório do desafio

Folhagem & Cia

Eu realizei este desafio com a ajuda do Codex, um assistente de inteligência artificial da OpenAI. Neste relatório, descrevo o que foi desenvolvido, como organizei o trabalho, as dificuldades que apareceram e como utilizei o assistente.

O que eu fiz

Eu parti do projeto-base com o CRUD de usuários e pedi ao assistente que me ajudasse a desenvolver a API de plantas da Folhagem & Cia. Com essa assistência, minha entrega passou a incluir o modelo Planta, o enum Luminosidade, a migration e as rotas de cadastro, listagem com filtros, consulta por id, edição parcial, exclusão, venda e cálculo do valor total do estoque.

Eu também incluí, com a ajuda do assistente, os quatro bônus: busca por nome sem diferenciar maiúsculas, ordenação por nome ou preço, listagem de plantas esgotadas e um script para carregar exemplos no banco.

Eu pedi que o assistente concluísse os requisitos e verificasse os casos de erro. A implementação aceita estoque zero e petFriendly false, rejeita números enviados como texto e valida os campos enviados na edição parcial. Para a venda, o assistente implementou uma atualização condicional dentro de uma transação, impedindo que compras simultâneas deixem o estoque negativo. Também pedi a preparação da documentação no arquivo API.md.

Como eu organizei o trabalho

Eu comecei fornecendo o enunciado e o repositório-base ao assistente. Primeiro, pedi ajuda para preparar o ambiente e executar o CRUD de usuários. Depois, solicitei a criação do modelo Planta e da migration. Em seguida, pedi que o assistente concluísse os repositórios, os controladores, as rotas, os bônus, os testes e a documentação.

Eu preferi manter o trabalho na branch main e optei por deixar os nomes das variáveis, funções, arquivos e pastas em português para facilitar meu entendimento e permitir uma verificação rápida do código. O assistente ajustou os imports, os testes e os comandos de execução, preservando os nomes exigidos pelo desafio e pelas bibliotecas. O modelo Usuario utiliza `@@map("User")` para manter a tabela existente sem alterar seus dados.

Na organização do meu projeto, mantive com a ajuda do assistente a separação entre rotas, controladores e repositórios. Os controladores validam os dados e definem as respostas HTTP; os repositórios consultam o banco com Prisma. Os preços ficam em centavos, e o cálculo do estoque utiliza BigInt internamente para evitar arredondamento antes da conversão para números JSON.

Eu solicitei o registro das alterações no Git. O histórico contém commits de implementação, documentação, tradução dos nomes e correções. A implementação principal ficou concentrada em um commit, em vez de ter um commit separado para cada etapa do enunciado.

Quais dificuldades eu encontrei

Minha primeira dificuldade foi configurar o Docker. Eu já havia tentado instalá-lo antes e encontrado um problema que associei à compatibilidade com o Windows. Pedi ao assistente que verificasse e configurasse o ambiente. Ele identificou a ausência do WSL, instalou o WSL 2 e o Docker Desktop e habilitou os componentes necessários. Eu reiniciei o computador para aplicar essas alterações.

Durante a preparação, o assistente encontrou o erro P1001 porque o PostgreSQL ainda estava inicializando na primeira tentativa de aplicar a migration. Depois que o banco ficou pronto, ele conseguiu aplicá-la. Para evitar esse problema nas próximas inicializações, o assistente adicionou uma verificação de prontidão do banco ao Docker Compose.

Eu também encontrei avisos no VS Code sobre os tipos do modelo Usuario, opções depreciadas do TypeScript e o endereço do schema do SWC. Enviei essas mensagens ao assistente, que regenerou o Prisma Client, ajustou as configurações e verificou a compilação e os testes. Ele também me orientou a selecionar o TypeScript do projeto e reiniciar o servidor TypeScript do editor para atualizar os avisos antigos.

Como eu utilizei IA

Eu utilizei o Codex como um assistente durante boa parte do desafio. Pedi ajuda para ler os requisitos, configurar o ambiente, gerar e editar código, corrigir erros, criar testes e preparar a documentação. O assistente executou os comandos e realizou as verificações técnicas; não apresento essas ações como testes que executei pessoalmente.

Para conferir a implementação, solicitei as verificações ao assistente e recebi os resultados da geração do Prisma Client, da aplicação das migrations, da checagem de tipos, da compilação e dos testes HTTP com PostgreSQL real. No teste de concorrência, o assistente enviou oito tentativas de venda para um único vaso: uma retornou 200, sete retornaram 400 e o estoque final ficou em zero.

Quais verificações eu solicitei

- Eu solicitei a aplicação da migration da Planta e a confirmação de que não havia migrations pendentes.
- Eu solicitei a checagem de tipos com `npm.cmd run typecheck`, que terminou sem erros.
- Eu solicitei a compilação com `npm.cmd run build`, que compilou os sete arquivos TypeScript.
- Eu solicitei os testes com `npm.cmd test`, que passaram com a API e o PostgreSQL reais, cobrindo cadastro, campos obrigatórios, valores inválidos, edição parcial, filtros, busca, ordenação, plantas esgotadas, totais, vendas simultâneas e exclusão.
- Eu solicitei a conferência dos totais com o banco vazio e o carregamento de exemplos pelo Docker e pelo Windows. O assistente confirmou os totais zerados e a permanência de três exemplos, sem duplicação.
- Eu solicitei a confirmação de que o PostgreSQL estava saudável e a API disponível na porta 3001.

Minha reflexão pessoal

Eu li todos os arquivos do projeto para acompanhar como o sistema está organizado. Entendi a rota como o endpoint que recebe uma chamada para executar determinada ação, como realizar uma venda ou cadastrar uma planta. Essa chamada é uma requisição, que pode trazer os dados necessários para a ação.

Na minha explicação inicial, associei o controlador à verificação dos campos obrigatórios, dos tipos e de como os dados devem ser utilizados. Com o ajuste técnico feito pelo assistente, descrevo seu papel de forma mais completa: o controlador recebe a requisição, valida os dados, chama o repositório e devolve uma resposta com o status HTTP correspondente.

Eu também descrevi o repositório como o lugar onde ficam os códigos e as configurações do sistema. Essa descrição corresponde ao repositório Git. Para explicar a organização interna da API, faço a distinção: a camada de repositório, como o arquivo RepositorioPlantas.ts, é responsável pelas consultas e alterações no banco por meio do Prisma. Assim, o caminho de uma requisição passa pela rota, pelo controlador, pelo repositório e pelo banco, até retornar uma resposta.

Minha dificuldade prática inicial foi colocar o Docker para funcionar, pois eu já havia tentado instalá-lo antes e encontrado problemas. Pedi ajuda ao assistente para preparar o ambiente e relatei os erros que apareceram depois. Na leitura do código, optei por nomes em português para facilitar meu entendimento e permitir uma verificação rápida das funções e variáveis.